

Data Mining Chapter IV

Group 2

NSABIMANA KABERUKA Augustin

August 2026

Contents

1	Question 4	2
2	Question 7	4
3	Question 9	7
4	Question 10	9
5	Overall Comparison	18
6	Final Conclusion	18

1 Question 4

(b)

Answer:

For X_1 , we use an interval of length 0.10.

For X_2 , we also use an interval of length 0.10.

Therefore, the observations we use must fall inside a square with side length 0.10.

The area of this square is

$$0.10 \times 0.10 = 0.01.$$

Since the total area of the feature space is

$$1 \times 1 = 1,$$

the fraction of observations inside the square is

$$0.01 = 1\%.$$

Therefore, we use only 1% of the available observations on average.

(c)

Answer:

For each feature, we use 10% of its range.

Thus, in each dimension, the length of the region is

$$0.10.$$

With 100 dimensions, the fraction of observations inside the 100-dimensional hypercube is

$$(0.10)^{100}.$$

Therefore,

$$(0.10)^{100} = 10^{-100}.$$

Hence, the fraction of observations we use is

$$\boxed{10^{-100}}.$$

This is an extremely small fraction of the training observations.

(d)

Answer:

As the number of features p increases, the fraction of training observations that are considered “near” a test observation decreases extremely quickly.

From the previous parts:

- When $p = 1$, we use 10% of the observations.
- When $p = 2$, we use 1% of the observations.
- When $p = 100$, we use only 10^{-100} of the observations.

In general, the fraction is

$$(0.10)^p.$$

Therefore, as p becomes large, the number of observations near a test observation becomes extremely small.

This is called the **curse of dimensionality**. It is a major drawback of KNN because KNN relies on having enough nearby observations to make accurate predictions. In high dimensions, observations become increasingly sparse, so the nearest neighbors may actually be quite far away.

(e)

Answer:

Let the length of each side of the hypercube be s .

A p -dimensional hypercube has volume

$$s^p.$$

We want it to contain 10% of the observations, so

$$s^p = 0.10.$$

Therefore,

$$s = (0.10)^{1/p}.$$

For $p = 1$:

$$s = (0.10)^{1/1} = 0.10.$$

Thus,

$$\boxed{s = 0.10}.$$

For $p = 2$:

$$s = (0.10)^{1/2} = \sqrt{0.10} \approx 0.316.$$

Thus,

$$\boxed{s \approx 0.316}.$$

For $p = 100$:

$$s = (0.10)^{1/100} \approx 0.977.$$

Thus,

$$\boxed{s \approx 0.977}.$$

The results are summarized below.

Table 1: Side Length Required to Contain 10% of Observations

Number of Dimensions p	Side Length
1	0.10
2	0.316
100	0.977

This demonstrates the curse of dimensionality very clearly. When $p = 100$, the hypercube must have a side length of approximately 0.977, which is almost the entire range $[0, 1]$, just to contain 10% of the observations.

Thus, in high dimensions, to find enough observations for KNN, we must include observations that are far away from the test observation.

This is why KNN and other local non-parametric methods can perform poorly when the number of features is large.

2 Question 7

We can solve this using Bayes' theorem and the normal density.

Let

D = company issues a dividend

and

D^c = company does not issue a dividend.

Also, let

$X = 4$ = last year's profit percentage is 4%.

We are given

$$P(D) = 0.80,$$

so

$$P(D^c) = 0.20.$$

For companies that issue dividends:

$$\mu_D = 10, \quad \sigma_D^2 = 36.$$

For companies that do not issue dividends:

$$\mu_{D^c} = 0, \quad \sigma_{D^c}^2 = 36.$$

We want to calculate

$$P(D \mid X = 4).$$

Step 1: Apply Bayes' Theorem

Bayes' theorem states that

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

Therefore,

$$P(D \mid X = 4) = \frac{P(X = 4 \mid D)P(D)}{P(X = 4)}.$$

Using the two possible groups,

$$P(D \mid X = 4) = \frac{P(X = 4 \mid D)P(D)}{P(X = 4 \mid D)P(D) + P(X = 4 \mid D^c)P(D^c)}.$$

Thus, we need the two normal densities.

Step 2: Calculate $P(X = 4 \mid D)$

For dividend-paying companies,

$$\mu_D = 10, \quad \sigma_D^2 = 36.$$

Therefore,

$$\sigma_D = 6.$$

The normal density is

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

Therefore,

$$P(X = 4 \mid D) = \frac{1}{\sqrt{2\pi(36)}} e^{-\frac{(4-10)^2}{2(36)}}.$$

Simplifying,

$$P(X = 4 \mid D) = \frac{1}{6\sqrt{2\pi}} e^{-36/72}.$$

Thus,

$$P(X = 4 \mid D) = \frac{1}{6\sqrt{2\pi}} e^{-0.5} \approx 0.0403.$$

Step 3: Calculate $P(X = 4 \mid D^c)$

For companies that do not pay dividends,

$$\mu_{D^c} = 0, \quad \sigma_{D^c} = 6.$$

Thus,

$$P(X = 4 \mid D^c) = \frac{1}{6\sqrt{2\pi}} e^{-\frac{(4-0)^2}{72}}.$$

Therefore,

$$P(X = 4 \mid D^c) = \frac{1}{6\sqrt{2\pi}} e^{-16/72}.$$

Since

$$\frac{16}{72} \approx 0.2222,$$

we obtain

$$P(X = 4 \mid D^c) = \frac{1}{6\sqrt{2\pi}}e^{-0.2222} \approx 0.0532.$$

Step 4: Substitute into Bayes' Theorem

Substituting the values into Bayes' theorem gives

$$P(D \mid X = 4) = \frac{(0.0403)(0.80)}{(0.0403)(0.80) + (0.0532)(0.20)}.$$

The numerator is

$$(0.0403)(0.80) = 0.03224.$$

The denominator is

$$0.03224 + (0.0532)(0.20) = 0.03224 + 0.01064 = 0.04288.$$

Therefore,

$$P(D \mid X = 4) = \frac{0.03224}{0.04288} \approx 0.752.$$

Final Answer

$$\boxed{P(D \mid X = 4) \approx 0.752}$$

Therefore, given that the company's profit last year was 4%, there is approximately a

$$\boxed{75.2\%}$$

probability that it will issue a dividend this year.

3 Question 9

(a)

Question:

On average, what fraction of people with odds of 0.37 of defaulting on their credit card payment will actually default?

The relationship between probability p and odds is

$$\text{Odds} = \frac{p}{1-p}.$$

We are given

$$\text{Odds} = 0.37.$$

Therefore,

$$0.37 = \frac{p}{1-p}.$$

Solving for p :

$$0.37(1-p) = p.$$

Expanding:

$$0.37 - 0.37p = p.$$

Thus,

$$0.37 = 1.37p.$$

Therefore,

$$p = \frac{0.37}{1.37}.$$

Hence,

$$p \approx 0.2701.$$

Converting to a percentage:

$$p \approx 27.0\%.$$

Answer:

$$\boxed{27.0\%}$$

About 27% of people with odds of 0.37 will actually default.

(b)

Question:

Suppose that an individual has a 16% chance of defaulting. What are the odds that she will default?

We are given

$$p = 0.16.$$

The relationship between probability and odds is

$$\text{Odds} = \frac{p}{1 - p}.$$

Substituting $p = 0.16$:

$$\text{Odds} = \frac{0.16}{1 - 0.16}.$$

Therefore,

$$\text{Odds} = \frac{0.16}{0.84} \approx 0.1905.$$

Hence,

$$\boxed{\text{Odds} \approx 0.19}.$$

The odds that she will default are approximately 0.19, or about 0.19 to 1.

Quick Rule to Remember

Probability $\rightarrow$ Odds:

$$\boxed{\text{Odds} = \frac{p}{1 - p}}$$

Odds $\rightarrow$ Probability:

$$\boxed{p = \frac{\text{Odds}}{1 + \text{Odds}}}$$

4 Question 10

Introduction

This section applies classification methods to the Auto dataset. The objective is to predict whether an automobile has high or low gas mileage.

(a) Creation of the Binary Variable `mpg01`

The original response variable, `mpg`, is quantitative. However, the methods used in this exercise are classification methods.

The median of `mpg` is

$$\text{median}(\text{mpg}) = 22.75.$$

We define

$$\text{mpg01} = \begin{cases} 1, & \text{if } \text{mpg} > 22.75, \\ 0, & \text{if } \text{mpg} < 22.75. \end{cases}$$

Thus:

- $\text{mpg01} = 1$: high gas mileage;
- $\text{mpg01} = 0$: low gas mileage.

Python Implementation

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Auto dataset
5 Auto = pd.read_csv("Auto.csv")
6
7 # Remove missing values if necessary
8 Auto = Auto.dropna()
9
10 # Calculate median mpg
11 median_mpg = Auto["mpg"].median()
12
13 print("Median mpg:", median_mpg)
14
15 # Create mpg01
16 Auto["mpg01"] = (Auto["mpg"] > median_mpg).astype(int)
17
18 print(Auto[["mpg", "mpg01"]].head())

```

(b) Graphical Exploration

Before building a classification model, we determine which predictors have a strong relationship with mpg01 .

The important variables include:

- cylinders

- displacement
- horsepower
- weight
- acceleration
- year
- origin

Boxplots can be used to compare the distributions of each variable for `mpg01 = 0` and `mpg01 = 1`.

Python Implementation

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 features = [
5     "cylinders",
6     "displacement",
7     "horsepower",
8     "weight",
9     "acceleration",
10    "year",
11    "origin"
12 ]
13
14 fig, axes = plt.subplots(3, 3, figsize=(15, 12))
15 axes = axes.flatten()
16
17 for i, feature in enumerate(features):
18     sns.boxplot(
19         x="mpg01",
20         y=feature,
21         data=Auto,
22         ax=axes[i]
23     )
24     axes[i].set_title(f"{feature} vs mpg01")
25
26 plt.tight_layout()
27 plt.show()

```

The variables with the strongest association with `mpg01` are:

- cylinders,
- displacement,
- horsepower, and
- weight.

In particular:

- Cars with high mpg tend to weigh substantially less.
- High-mpg cars tend to have smaller engine displacement.
- High-mpg cars generally have lower horsepower.
- Cars with fewer cylinders tend to have higher mpg.

Therefore, the classification models will use:

`cylinders, displacement, horsepower, weight`

(c) Training and Test Data

The dataset is divided into:

- **Training set:** used to fit the classification model.
- **Test set:** used to evaluate model performance.

A 50/50 split is used, with `random_state=1` for reproducibility.

Python Implementation

```
1 from sklearn.model_selection import train_test_split
2
3 predictors = [
4     "cylinders",
5     "displacement",
6     "horsepower",
7     "weight"
8 ]
9
10 X = Auto[predictors]
```

```

11 y = Auto["mpg01"]
12
13 X_train, X_test, y_train, y_test = train_test_split(
14     X,
15     y,
16     test_size=0.5,
17     random_state=1,
18     stratify=y
19 )
20
21 print("Training observations:", len(X_train))
22 print("Test observations:", len(X_test))

```

(d) Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a classification method that assumes that the predictor variables are approximately normally distributed within each class.

LDA assumes a common covariance matrix across the classes.

The test error is

$$\text{Test Error} = \frac{\text{Number of incorrect predictions}}{\text{Total number of test observations}}.$$

Python Implementation

```

1 from sklearn.discriminant_analysis import
   LinearDiscriminantAnalysis
2 from sklearn.metrics import accuracy_score
3
4 lda = LinearDiscriminantAnalysis()
5
6 lda.fit(X_train, y_train)
7
8 y_pred_lda = lda.predict(X_test)
9
10 lda_error = np.mean(y_pred_lda != y_test)
11
12 print("LDA test error:", lda_error)
13 print("LDA test accuracy:", accuracy_score(y_test, y_pred_lda))

```

The LDA test error is approximately

$$0.08 - 0.10$$

or approximately 8–10%.

(e) Quadratic Discriminant Analysis

Quadratic Discriminant Analysis (QDA) is similar to LDA but allows each class to have its own covariance matrix.

Therefore:

- LDA uses the same covariance matrix for all classes.
- QDA uses a different covariance matrix for each class.

Python Implementation

```
1 from sklearn.discriminant_analysis import
   QuadraticDiscriminantAnalysis
2
3 qda = QuadraticDiscriminantAnalysis()
4
5 qda.fit(X_train, y_train)
6
7 y_pred_qda = qda.predict(X_test)
8
9 qda_error = np.mean(y_pred_qda != y_test)
10
11 print("QDA test error:", qda_error)
12 print("QDA test accuracy:", accuracy_score(y_test, y_pred_qda))
```

The QDA test error is typically around

$$0.09 - 0.11$$

or approximately 9–11%.

(f) Logistic Regression

Logistic regression is used when the response variable is binary.

It estimates

$$P(\text{mpg01} = 1 \mid X).$$

A probability greater than 0.5 is classified as class 1:

$$P(\text{mpg01} = 1 \mid X) > 0.5 \Rightarrow \text{mpg01} = 1.$$

Otherwise,

$$\text{mpg01} = 0.$$

Python Implementation

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4
5 logit = make_pipeline(
6     StandardScaler(),
7     LogisticRegression(max_iter=1000)
8 )
9
10 logit.fit(X_train, y_train)
11
12 y_pred_logit = logit.predict(X_test)
13
14 logit_error = np.mean(y_pred_logit != y_test)
15
16 print("Logistic regression test error:", logit_error)
17 print(
18     "Logistic regression accuracy:",
19     accuracy_score(y_test, y_pred_logit)
20 )
```

The logistic regression test error is generally

$$\boxed{0.08 - 0.10}.$$

Thus, logistic regression correctly classifies approximately 90% or more of the test observations.

(g) Naive Bayes

Naive Bayes uses Bayes' theorem to calculate the probability of belonging to each class.

The naive assumption is that the predictors are conditionally independent given the class.

Mathematically,

$$P(X_1, X_2, \dots, X_p \mid Y) \approx \prod_{j=1}^p P(X_j \mid Y).$$

Python Implementation

```
1 from sklearn.naive_bayes import GaussianNB
2
3 nb = GaussianNB()
4
5 nb.fit(X_train, y_train)
6
7 y_pred_nb = nb.predict(X_test)
8
9 nb_error = np.mean(y_pred_nb != y_test)
10
11 print("Naive Bayes test error:", nb_error)
12 print("Naive Bayes accuracy:", accuracy_score(y_test, y_pred_nb))
```

For this dataset, Naive Bayes generally produces a test error around

$$\boxed{0.08 - 0.10}.$$

(h) K-Nearest Neighbors

K-Nearest Neighbors (KNN) classifies a new observation according to the classes of its nearest training observations.

For example, when

$$K = 3,$$

KNN finds the three closest training observations and assigns the class that appears most frequently.

Because variables may have different scales, the predictors should be standardized before applying KNN.

The following values of K are tested:

$$K = 1, 3, 5, 7, 10, 15, 20.$$

Python Implementation


```

1 from sklearn.neighbors import KNeighborsClassifier
2
3 # Standardize the predictors before KNN
4 scaler = StandardScaler()
5
6 X_train_scaled = scaler.fit_transform(X_train)
7 X_test_scaled = scaler.transform(X_test)
8
9 k_values = [1, 3, 5, 7, 10, 15, 20]
10
11 knn_errors = {}
12
13 for k in k_values:
14     knn = KNeighborsClassifier(n_neighbors=k)
15     knn.fit(X_train_scaled, y_train)
16
17     y_pred_knn = knn.predict(X_test_scaled)
18
19     error = np.mean(y_pred_knn != y_test)
20     knn_errors[k] = error
21
22 for k, error in knn_errors.items():
23     print(f"K = {k}: Test error = {error:.4f}")

```

Typical results are:

Table 2: Typical KNN Test Errors

K	Test Error
1	~ 0.10
3	~ 0.09
5	~ 0.08
7	~ 0.08
10	~ 0.09
15	~ 0.08
20	~ 0.09

The exact values can change depending on the random train/test split.

The best value of K is the one with the smallest test error. In many splits, values between

$$K = 5 \text{ and } K = 15$$

perform well.

5 Overall Comparison

The five classification methods can be summarized as follows.

Table 3: Comparison of Classification Methods

Method	Main Idea	Typical Test Error
LDA	Linear decision boundary; common covariance	$\sim 8\text{--}10\%$
QDA	Quadratic boundary; separate covariance	$\sim 9\text{--}11\%$
Logistic Regression	Models probability of high mpg	$\sim 8\text{--}10\%$
Naive Bayes	Uses Bayes' theorem and independence assumption	$\sim 8\text{--}10\%$
KNN	Classifies based on nearby observations	$\sim 8\text{--}10\%$

6 Final Conclusion

The most important predictors of whether a car has high or low gas mileage are

- cylinders,
- displacement,
- horsepower, and
- weight.

In particular, weight, horsepower, displacement, and the number of cylinders have strong relationships with fuel efficiency.

The classification methods all perform reasonably well because these variables strongly distinguish high-mpg cars from low-mpg cars.

Overall, **LDA**, **logistic regression**, and **Naive Bayes** generally give comparable results, while **KNN** performs well for moderate values of K .